BrainCode Academy

# pandas: анализ городских поездок

Неделя 03. Вторник, 90 минут
Индекс, фильтрация, преобразования, groupby

Assylbek Gizatov

Machine learning and Artificial Intelligence

# Что мы сделаем за занятие

Разберём таблицу поездок: одна строка, один запрос на поездку.

Выберем нужные записи с loc и iloc, проверим фильтры.

Создадим календарные признаки и рассчитаем метрики.

Получим проверяемый отчёт о статусах, длительности и спросе.

# План на 90 минут

| Время | Формат | Содержание |
|---|---|---|
| 0–18 | Теория | Таблица, типы, индекс и выборка |
| 18–45 | Практика | Загрузка, аудит, loc/iloc, маски |
| 45–63 | Теория | Признаки, агрегаты, знаменатели |
| 63–90 | Практика | groupby, задания, проверка результата |

36 минут теории и 54 минуты практики. Справочные слайды в конце можно изучить после занятия.

# Учебный кейс: городские поездки

Команде городского транспортного сервиса нужен отчёт: когда поступают запросы и где поездки отменяются чаще.

Поля времени, статуса, длительности и стоимости позволяют построить базовые признаки для будущей ML-задачи.

Все записи синтетические. Они не описывают реальных пассажиров, тарифы или транспорт конкретного города.

# Паспорт городского набора

| Свойство | Значение |
| --- | --- |
| CSV до очистки | 170 строк, 8 полей |
| После очистки | 168 уникальных заявок |
| Период | 5–11 октября 2026, 7 условных дней |
| Справочник | 5 зон, один неизвестный ключ Z99 |
| Конструкция времени | 24 заявки на день, каждые 30 минут с 06:00 |

Одинаковое число заявок задано при создании примера. Здесь нельзя обнаружить реальный час пик.

# Фрагмент trips.csv

| trip_id | requested_at | zone | status | fare, KZT |
|---|---|---|---|---|
| 1001 | 10-05 06:00 | Z01 | cancelled | — |
| 1002 | 10-05 06:30 | Z03 | completed | 1 100 |
| 1003 | 10-05 07:00 | Z02 | completed | 3 450 |
| 1004 | 10-05 07:30 | Z03 | completed | 2 650 |

У cancelled отсутствуют фактическая дистанция, длительность и оплата. Это ожидаемые пропуски, а не нулевая поездка.

# Единица наблюдения и вопрос анализа

Единица наблюдения: один запрос на поездку с уникальным trip_id.

Число запросов включает отмены. Длительность и стоимость завершённых поездок считаем на другой выборке.

До написания кода формулируем вопрос: какие строки входят в метрику и что означает её знаменатель?

# DataFrame и Series

DataFrame: двумерная таблица. У неё есть индекс строк и названия столбцов.

Series: один столбец значений со своим индексом. Операции над Series обычно работают сразу со всеми строками.

Столбцы могут иметь разные dtype. Время, текст, деньги и идентификаторы требуют разных проверок.

# Одна колонка и таблица из одной колонки

```
s = trips["status"]
one_column = trips[["status"]]
print(type(s).__name__, s.ndim)
print(type(one_column).__name__, one_column.ndim)
```

Одинарный выбор даёт Series, двойные квадратные скобки дают DataFrame.

Результат: Series 1 и DataFrame 2. Это влияет на форму следующих операций.

# Типы данных и смысл полей

trip_id и zone_id: ключи. Сумма или среднее ID не имеют аналитического смысла.

requested_at: дата и время создания заявки. Это не фактическое начало поездки. datetime даёт час и день недели.

duration_min: измерение в минутах. Пропуск для отмены нельзя автоматически заменить нулём.

fare_kzt: стоимость в KZT. Сначала определяем, для каких статусов она имеет смысл.

# Чтение файлов из учебного notebook

```
import pandas as pd
# DATA_DIR задаёт первый блок учебного notebook
raw = pd.read_csv(DATA_DIR / "trips.csv",
                  parse_dates=["requested_at"])
raw.head()
```

Сначала выполняем блок подготовки в Week_03_City_Trips_Pandas.ipynb. Он создаёт отдельную папку и два файла без сети.

Внешний файл CSV устроен так же. Меняется только путь, переданный read_csv.

# Первый осмотр таблицы

```
print(raw.shape)
print(raw.columns.tolist())
print(raw.dtypes)
raw.info()
raw.isna().sum()
```

shape, columns и dtypes являются атрибутами. info() и isna() вызываем как методы.

Проверяем схему, число строк, тип времени и расположение пропусков до любых преобразований.

# Полный дубль и конфликт ключа

Полный дубль: совпадают все поля строки. В учебной выгрузке такие строки добавлены намеренно.

Конфликт ключа: одинаковый trip_id, но разные значения других полей. Автоматически выбирать одну запись опасно.

Сначала смотрим повторы. После удаления полных дублей отдельно проверяем уникальность ключа.

# Очистка только точных дублей

```
duplicate_rows = raw.duplicated().sum()
conflicts = raw.loc[raw.duplicated("trip_id", keep=False)]
trips = raw.drop_duplicates().copy()
assert trips["trip_id"].is_unique
print(duplicate_rows, len(trips))
```

duplicated() сравнивает все поля. duplicated("trip_id") сравнивает только ключ.

copy() явно отделяет рабочую таблицу. raw остаётся исходной поставкой для аудита.

# Результат первичного аудита

| Проверка | До очистки | После очистки |
|---|---|---|
| Строки | 170 | 168 |
| Полные повторы сверх первой строки | 2 | 0 |
| Уникальные trip_id | 168 | 168 |
| completed / cancelled | 140 / 30 | 139 / 29 |

Полные дубли: trip_id 1009 (cancelled) и 1044 (completed). Копия 1044 добавила бы лишние 3 050 KZT.

# Пропуски после удаления дублей

| Поле | Пропусков | Как трактуем |
|---|---|---|
| channel | 3 | Канал неизвестен, заявки сохраняем |
| distance_km | 29 | У отмен нет фактической дистанции |
| duration_min | 31 | 29 отмен + 2 completed без измерения |
| fare_kzt | 29 | У cancelled нет оплаты |

Для каждого столбца действует своё правило. Единое fillna(0) исказит смысл и категории, и измерения.

# Метки и позиции строк

## loc: labels

trips.loc[mask, columns]

Работает по меткам индекса и названиям столбцов.

Срез меток включает правую границу, если она найдена.

## iloc: integer positions

trips.iloc[:3, :4]

Работает по порядковым номерам строк и столбцов.

Правая граница среза исключается, как у списка Python.

# Позиционная выборка iloc

```
trips.iloc[:3, :4]
trips.iloc[[0, 2, 4], [0, 1, 3]]
trips.iloc[-1]
```

Первая строка выбирает первые 3 строки и первые 4 столбца.

Номера в iloc зависят от текущего порядка. После сортировки позиция 0 может означать другую поездку.

# Бизнес-ключ как индекс

```
indexed = trips.set_index("trip_id", verify_integrity=True)
first_id = indexed.index[0]
indexed.loc[first_id, ["status", "fare_kzt"]]
restored = indexed.reset_index()
```

verify_integrity=True требует уникальность нового индекса. Сам по себе индекс pandas может содержать повторы.

reset_index() возвращает ключ в столбец. Данные сохраняются, меняется способ адресации.

# Выбор по метке и по позиции

```
demo = pd.DataFrame({"value": [12, 25, 40]},
                    index=[10, 20, 30])
print(demo.loc[10:20, "value"].tolist())
print(demo.iloc[0:2, 0].tolist())
```

Оба выражения здесь дают [12, 25], но по разным правилам.

demo.loc[0] ищет метку 0 и вызовет KeyError. demo.iloc[0] выбирает первую строку.

# Булева маска

Сравнение столбца с условием возвращает Series из True и False.

Для каждой строки True означает «оставить», False означает «не включать в выборку».

Проверяем число выбранных строк и само условие. Успешное выполнение кода ещё не доказывает правильность фильтра.

# Фильтрация завершённых поездок

```
mask = trips["status"].eq("completed")
completed = trips.loc[mask].copy()
print(mask.sum(), len(completed))
assert completed["status"].eq("completed").all()
```

eq("completed") эквивалентно сравнению == "completed".

sum() булевой маски считает True. copy() удобен, если дальше меняем отдельную выборку.

# Несколько условий: &, | и ~

```
mask = (trips["status"].eq("completed")) & (
    trips["duration_min"].ge(30)
)
long_trips = trips.loc[mask, ["trip_id", "duration_min"]]
not_cancelled = trips.loc[~trips["status"].eq("cancelled")]
```

Каждое условие заключаем в скобки. & означает «и», | означает «или», ~ инвертирует маску.

and/or работают с отдельными булевыми значениями и не заменяют операторы для Series.

# Списки значений и пропуски

```
selected = trips.loc[trips["zone_id"].isin(["Z01", "Z02"])]
known = trips.loc[trips["duration_min"].notna()]
missing = trips.loc[trips["duration_min"].isna()]
counts = trips["status"].value_counts(dropna=False)
```

isin задаёт принадлежность набору. isna/notna проверяют пропуск независимо от его внутреннего представления.

Сравнение с NaN через == не является корректной проверкой отсутствующего значения.

# Сортировка и выбор первых строк

```
top = completed.sort_values(
    ["fare_kzt", "trip_id"], ascending=[False, True]
).head(5)
top[["trip_id", "fare_kzt", "duration_min"]]
```

Сначала сортируем стоимость по убыванию. trip_id задаёт воспроизводимый порядок при равных ценах.

head(5) без предварительной сортировки означает первые записи файла, а не пять самых дорогих поездок.

# Практика 1: выборка поездок

За 7 минут: загрузите CSV, посчитайте полные дубли и получите trips с уникальным trip_id.

Выберите завершённые поездки длительностью не менее 30 минут. Оставьте ID, зону, длительность и стоимость.

Отсортируйте результат по длительности по убыванию. Докажите условия через assert.

Обсуждение: почему позиционный фильтр «первые 20 строк» не отвечает на этот вопрос?

# Разбор практики 1

```
mask = trips["status"].eq("completed") & (
    trips["duration_min"].ge(30)
)
answer = trips.loc[mask, ["trip_id", "zone_id",
                          "duration_min", "fare_kzt"]]
answer = answer.sort_values("duration_min", ascending=False)
assert answer["duration_min"].ge(30).all()
```

Дополнительно проверяем, что каждый trip_id из answer относится к completed в исходной таблице.

Количество строк и первые результаты сравниваем с выводом notebook.

# Фильтр: точный результат на данных

| trip_id | Зона | Минуты | Оплата, KZT |
|---|---|---|---|
| 1022 | Z02 | 64.8 | 3 650 |
| 1068 | Z05 | 62.7 | 3 850 |
| 1058 | Z04 | 61.1 | 3 800 |
| 1144 | Z03 | 60.0 | 3 800 |
| 1006 | Z05 | 59.4 | 3 150 |

Условию completed и duration_min ≥ 30 соответствуют 72 поездки. Показаны 5 самых длительных.

# Производные признаки

Признак можно вычислить из существующих столбцов: час запроса, день недели, отметка завершения.

Векторные операции pandas работают над Series целиком. Для этих вычислений цикл по строкам не нужен.

Новый признак должен иметь понятные единицы и правило обработки неизвестных значений.

# assign и календарные признаки

```
trips = trips.assign(
    request_hour=trips["requested_at"].dt.hour,
    weekday=trips["requested_at"].dt.dayofweek,
    is_completed=trips["status"].eq("completed"),
    is_cancelled=trips["status"].eq("cancelled")
)
```

dt.dayofweek использует номера от 0 (понедельник) до 6 (воскресенье).

assign возвращает новую таблицу. Здесь мы явно сохраняем её обратно в trips.

# Учебная метрика: сумма оплат

```
trips["payments_kzt"] = trips["fare_kzt"].where(
    trips["is_completed"], 0
)
assert trips.loc[~trips["is_completed"],
                 "payments_kzt"].eq(0).all()
```

payments_kzt оставляет fare_kzt у completed и ставит 0 у cancelled. В notebook сумму также считаем напрямую по completed.

Это сумма оплат в синтетическом наборе. Она не является прибылью. Комиссии и расходы не моделируются.

# Ноль и неизвестное значение

## Сумма по принятому правилу

Для отменённой заявки payments_kzt = 0 по принятому определению суммы.

Правило явно определяет отсутствие вклада в общую сумму.

## Измерение длительности

У отмен duration_min отсутствует. Кроме того, у 2 completed длительность неизвестна.

Ноль означал бы известную поездку длительностью 0 минут и исказил бы среднее.

# Изменение через loc без цепочек

```
trips["fare_band"] = "unknown"
trips.loc[trips["fare_kzt"].notna(), "fare_band"] = "standard"
trips.loc[trips["fare_kzt"].ge(3000), "fare_band"] = "high"
print(trips["fare_band"].value_counts())
```

Одна операция loc одновременно задаёт строки и столбец для записи.

Цепочка trips[mask]["fare_band"] = ... может менять временный объект и не даёт надёжной семантики записи.

# GroupBy: split, apply, combine

Split: pandas разделяет строки по ключам группировки, например status или request_hour.

Apply: вычисляет агрегат в каждой группе, например число строк, сумму или медиану.

Combine: собирает результаты в таблицу, где одна строка обычно соответствует одной группе.

После группировки меняется единица наблюдения. Теперь это группа, а не отдельная поездка.

# Именованные агрегаты

```
status_report = trips.groupby("status").agg(
    requests=("trip_id", "size"),
    known_duration=("duration_min", "count"),
    median_duration=("duration_min", "median"),
    payments_kzt=("payments_kzt", "sum")
)
status_report
```

Слева задаём понятное имя результата. Справа указываем исходный столбец и агрегат.

Медиана длительности использует только непустые наблюдения. Число наблюдений показываем рядом.

# size и count

## size

Считает строки группы.

requests=("trip_id", "size") отвечает: сколько запросов пришло?

## count

Считает непустые значения выбранного поля.

known_duration=("duration_min", "count") отвечает: сколько длительностей мы знаем?

# Почему число измерений меньше числа поездок

| Статус | Заявки | Длительность известна | Медиана, мин |
|---|---|---|---|
| cancelled | 29 | 0 | — |
| completed | 139 | 137 | 30.2 |

29 отмен не имеют длительности по смыслу. Ещё у 2 завершённых поездок длительность отсутствует: это отдельный риск качества.

# Средняя длительность только completed

```
completed = trips.loc[trips["is_completed"]].copy()
by_hour = completed.groupby("request_hour").agg(
    completed_trips=("trip_id", "size"),
    known_duration=("duration_min", "count"),
    mean_minutes=("duration_min", "mean"),
    median_minutes=("duration_min", "median"),
    payments_kzt=("payments_kzt", "sum")
)
```

Небольшое число поездок делает сравнение часов неустойчивым. Всегда оставляем размер группы.

Разница средних в учебном наборе не доказывает причинное влияние часа на длительность.

# Доля отмен и её знаменатель

```
demand = trips.groupby("request_hour").agg(
    requests=("trip_id", "size"),
    cancellations=("is_cancelled", "sum"),
    cancellation_rate=("is_cancelled", "mean")
)
demand["cancellation_pct"] = 100 * demand["cancellation_rate"]
```

Среднее булева столбца равно доле True. Знаменатель здесь: все запросы данного часа.

Доля 0.20 и 20% обозначают одно число. Умножаем на 100 только при представлении в процентах.

# Контрольные суммы

Число запросов в группах должно совпасть с числом строк trips.

Сумма payments_kzt после группировки должна совпасть с исходной суммой.

Средние и доли нельзя складывать. Для общей средней длительности веса равны числу известных длительностей в группах.

Контроль ловит потерянные строки, дублирование и несогласованный фильтр.

# Проверки результата группировки

```
assert status_report["requests"].sum() == len(trips)
assert status_report["payments_kzt"].sum() == trips["payments_kzt"].sum()
assert demand["requests"].sum() == len(trips)
assert demand["cancellation_rate"].between(0, 1).all()
```

Каждый assert проверяет конкретное обещание отчёта. Ошибка останавливает расчёт раньше публикации.

Для вычислений с дробными числами вместо строгого равенства обычно нужен допуск, например np.isclose.

# Контрольные числа для всей группы

168 заявок: 139 completed и 29 cancelled.

Доля отмен: 17.26%. Сумма оплат completed: 308 400 KZT.

Известны 137 длительностей completed. Средняя: 31.76 мин, медиана: 30.2 мин.

Эти результаты относятся к фиксированным синтетическим записям и служат для сверки кода.

# Практика 2: отчёт по дням недели

За 12 минут: получите weekday, число запросов, число completed и долю отмен для каждого дня недели.

Для completed добавьте среднюю и медианную длительность. Не заменяйте пропуски нулями.

Сверьте число запросов с trips. Сформулируйте 2 вывода с числом наблюдений.

Вопрос на обсуждение: одинаково ли много понедельников и воскресений попало в период выгрузки?

# Разбор практики 2: базовый отчёт

```
weekday_report = trips.groupby("weekday").agg(
    requests=("trip_id", "size"),
    completed=("is_completed", "sum"),
    cancel_rate=("is_cancelled", "mean")
)
assert weekday_report["requests"].sum() == len(trips)
```

Длительности completed можно агрегировать отдельно, затем соединить по weekday.

Число календарных дат следует считать по уникальным датам, а не по числу строк запросов.

# Итог вторника

Вы умеете различать Series и DataFrame, метку и позицию, полный дубль и конфликт ключа.

Можете написать проверяемый фильтр, создать календарные признаки и сгруппировать таблицу.

Понимаете, почему count отличается от size, а ноль отличается от неизвестного значения.

В четверг добавим справочник зон, проверим merge и сохраним отчёт.

# Самопроверка перед четвергом

Почему iloc[:3] возвращает три строки, а loc[10:20] не означает «десять строк»?

Как выбрать completed с duration_min ≥ 30 без and?

Как проверить, что группировка сохранила все запросы?

Почему среднее payments_kzt по всем заявкам отличается от среднего fare_kzt по completed?

# Справочник: основные операции

Осмотр: head(), info(), shape, dtypes, isna().sum().

Выборка: loc, iloc, isin, between, isna, notna.

Преобразования: assign, .loc[...] = ..., where, .dt.hour.

Агрегаты: groupby().agg(), size, count, sum, mean, median.

# Источники и работа после занятия

Учебный notebook Week_03_City_Trips_Pandas.ipynb создаёт trips.csv и zones.json в первом блоке. Таблицы в этой презентации используют те же записи.

Официальные объяснения API: pandas User Guide, Indexing and selecting data, GroupBy.

Недельный план: 4 академических часа, два занятия по 90 минут. Порядок внутри уроков является методической организацией.

Дополнительные материалы в presentations/ расширяют тему. Все численные выводы в этом deck относятся к синтетическому набору.